Gift Tracker Project: Conversation with Claude

CISC final project (Drafter website). Transcript of the chat. Long code listings that were repeated in the chat are abridged where marked; the complete final code is in the last Claude message.

You

Hi, I need help creating a "route" for my final project in python coding CISC class

You

its the index route that will take in my state and create a page called "Gift Tracker", but im not sure how to do that, here is my State class:

```
@dataclass
class State:
    overall_budget:float
    total_spent:float
    reciprients:list[People]
```

Claude

Here's a starting point for the index route:

```
from drafter import *
from dataclasses import dataclass

@dataclass
class State:
    overall_budget: float
    total_spent: float
    recipients: list[People]

@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Gift Tracker"),
        "Total budget: $" + str(state.overall_budget),
        "Spent so far: $" + str(state.total_spent),
        "Money left: $" + str(state.overall_budget - state.total_spent),
        Button("Add a gift", add_gift_page)
    ])

start_server(State(0.0, 0.0, []))
```

Key points: - A route takes `state` and returns a `Page(state, [list of content])`. - `Header("Gift Tracker")` is what puts the title on the page. - Content strings must be strings, so use `str(...)` on numbers. - `Button("text", function_name)` takes you to another route, so you'll need to define `add_gift_page` too. - I changed `reciprients` to `recipients`. Whatever spelling you pick, use it everywhere.

Want to do the `add_gift_page` route next?

You

```
@route
def view_people(state: State) -> Page:
    return Page(state, [
        Header("People"),
        Button("Back", index),
        Button("Add Reciprient", add_reciprient)
    ])
```

this is my view_people route im working with, it works so far but Im not sure how to get my "add_reciprient" function to actually add someone to the view_people page

Claude

You need two pieces: a form page, and a route that adds the person to `state.recipients`, then shows `view_people`.

```
@route
```

```
def add_recipient(state: State) -> Page:
    return Page(state, [
        Header("Add Recipient"),
        "Name:",
        TextBox("name"),
        Button("Save", save_recipient),
        Button("Cancel", view_people)
    ])

@route
def save_recipient(state: State, name: str) -> Page:
    state.recipients.append(People(name, []))
    return view_people(state)
```

Then update `view_people` to display everyone in the list:

```
@route
def view_people(state: State) -> Page:
    content = [Header("People")]
    for person in state.recipients:
        content.append(person.name)
    content.append(Button("Add Recipient", add_recipient))
    content.append(Button("Back", index))
    return Page(state, content)
```

Key points: - The `TextBox("name")` name must match the parameter name `name` in `save_recipient`. That's how Drafter passes the typed text in. - `People(name, [])` assumes your dataclass is `People(name, gifts)`. Adjust it to match yours. - `return view_people(state)` calls the route directly so the page refreshes with the new person.

You

how would the textbox actually save

Claude

The textbox doesn't save anything on its own. The **button** does the work.

When the user clicks `Button("Save", save_recipient)`, Drafter looks at the page, grabs whatever is typed in each component, and passes those values into `save_recipient` as arguments. It matches them by name:

```
TextBox("name")                         # the box is labeled "name"
def save_recipient(state, name: str):   # parameter is also "name"
```

So `name` inside `save_recipient` is whatever the user typed. From there, you save it yourself by adding it to the state:

```
state.recipients.append(People(name, []))
```

That line is the actual saving. Because `state` gets passed along to every page, the new person is still there when `view_people` runs.

If the names don't match (like `TextBox("person")` but a parameter called `name`), you'll get an error, so double-check that first if it breaks.

You

```
@route
def view_people(state: State) -> Page:
    content = [Header("People")]
    for person in state.recipients:
        content.append(Button(person.name, view_person, [Argument("person_name", person.name)]))
    content.append(Button("Add Recipient", add_recipient))
    content.append(Button("Back", index))
    return Page(state, content)

@route
def view_person(state: State, person_name: str) -> Page:
    for person in state.recipients:
        if person.name == person_name:
            content = [Header(person.name + "'s Gifts")]
    return view_people(state)
```

I wanted each persons name to be a button so I tried making a new route called view_person which shows the individual persons list, but its telling me theres an error

Claude

Your `view_person` builds `content` but never uses it. It always ends with `return view_people(state)`, so you never see the person's page. And if no name matched, `content` wouldn't exist at all, which can cause an error.

Return the `Page` inside the loop:

```
@route
def view_person(state: State, person_name: str) -> Page:
    for person in state.recipients:
        if person.name == person_name:
            content = [Header(person.name + "'s Gifts")]
            for gift in person.gifts:
                content.append(gift.name + ": $" + str(gift.price))
            content.append(Button("Back", view_people))
            return Page(state, content)
    return view_people(state)
```

Key points: - The `return Page(...)` inside the `if` is what shows the person's page. - The final `return view_people(state)` is only a fallback if no match is found. - `gift.name` and `gift.price` assume those are your `Gift` fields. Change them to match your dataclass. - Your `Button(person.name, view_person, [Argument(...)])` looks right.

If you still get an error, paste the exact message and I'll help track it down.

You

I created two more routes :

```
@route
def add_gift(state: State):
   return Page(state, [
        Header("Add Gift"),
        "Gift Name:",
        TextBox("name"),
        TextBox("price"),
        Button("Save", save_gift),
        Button("Cancel", view_person)
    ])

@route
def save_gift(state: State, name:str, price:float):
    state.recipients.gifts.append(Gift(name, price, False))
    return view_people(state)
```

by copying the add reciprient and save reciprient functions, but I dont know how to connect them to the view_person route

Claude

You need to pass the person's name along the chain: `view_person` -> `add_gift` -> `save_gift`. Buttons can carry values with `Argument`, just like you did in `view_people`.

1. In `view_person`, add a button that sends the name:

```
content.append(Button("Add Gift", add_gift, [Argument("person_name", person.name)]))
```

2. `add_gift` receives the name and passes it on:

```
@route
def add_gift(state: State, person_name: str) -> Page:
    return Page(state, [
        Header("Add Gift for " + person_name),
        "Gift Name:",
        TextBox("name"),
        "Price:",
        TextBox("price"),
        Button("Save", save_gift, [Argument("person_name", person_name)]),
        Button("Cancel", view_person, [Argument("person_name", person_name)])
    ])
```

3. `save_gift` finds the right person:

```
@route
def save_gift(state: State, name: str, price: float, person_name: str) -> Page:
    for person in state.recipients:
        if person.name == person_name:
            person.gifts.append(Gift(name, price, False))
            state.total_spent = state.total_spent + price
    return view_person(state, person_name)
```

Key points: - `state.recipients` is a list, so it has no `.gifts`. Each *person* does, which is why you loop to find the right one. - Your Cancel button needed an `Argument` too, because `view_person` requires `person_name`. - Ending with `view_person(state, person_name)` returns you to that person's gift list. - I added the `total_spent` update since your State tracks it. Remove it if you'd rather handle that later.

You

[You pasted your full project code at this point (dataclasses Gift, Person, State and the routes index, save_budget, add_reciprient, save_reciprient, view_people, view_person, add_gift, save_gift). Omitted here; the final version is at the end.]

THIS IS MY CODE

I am just confused on how I am supposed to make the gifts checked off if they are bought im trying the use the checklist feature in drafter, but its not really working. Also I dont understand how to keep track of how much the gifts cost and if we are still under the total budget saved at the beginning

Claude

The checkbox is tricky here because each checkbox needs its own fixed parameter name, and your number of gifts changes. A simpler fix is a toggle button on each gift.

1. Checklist: in `view_person`, replace the gift loop:

```
for gift in person.gifts:
    mark = "[x] " if gift.is_bought else "[ ] "
    content.append(mark + gift.name + ": $" + str(gift.price))
    content.append(Button("Toggle bought", toggle_bought,
                          [Argument("person_name", person.name),
                           Argument("gift_name", gift.name)]))

@route
def toggle_bought(state: State, person_name: str, gift_name: str) -> Page:
    for person in state.recipients:
        if person.name == person_name:
            for gift in person.gifts:
                if gift.name == gift_name:
                    gift.is_bought = not gift.is_bought
    return view_person(state, person_name)
```

2. Budget: use the sum pattern instead of adding as you go:

```
def total_gifts(gifts: list[Gift]) -> float:
    total = 0.0
    for gift in gifts:
        total = total + gift.price
    return total

def total_all(people: list[Person]) -> float:
    total = 0.0
    for person in people:
        total = total + total_gifts(person.gifts)
    return total
```

In `save_gift`, replace the `total_spent` line with this after the loop:

```
state.total_spent = total_all(state.recipients)
```

3. Over-budget warning in `index`: build the content as a list, like you did in `view_people`:

```
content = [Header("Gift Tracker"), ...your existing lines...]
if state.total_spent > state.budget:
    content.append("Warning: over budget by $" + str(state.total_spent - state.budget))
return Page(state, content)
```

Key points: - Recalculating with the sum pattern keeps the total correct, and it matches what your design recipe expects. - Your `Person.total` field isn't used yet. You can use `total_gifts(person.gifts)` on each person's page instead. - Gift names need to be unique per person for the toggle to find the right one.

You

well now It wont show me anyones list of gifts when i click on their button

Claude

I can't tell the cause without seeing what changed. Two things would pinpoint it:

1. The exact error message (Drafter usually shows it on the page or in the console).
2. Your current `view_person` code after the edit.

In the meantime, check these common causes:

- **Indentation:** the `return Page(state, content)` must stay inside the `if person.name == person_name:` block, and the new `mark = ...` lines must be inside the `for gift in person.gifts:` loop.
- `toggle_bought` **is missing or misspelled:** a button pointing to a function that doesn't exist will crash the whole page.
- **Argument names:** `Argument("person_name", ...)` and `Argument("gift_name", ...)` must exactly match the parameter names in `toggle_bought`.
- **Old saved data:** if you changed your dataclasses and kept the same running app, restart it so it starts from a fresh `State(0.0, 0.0, [])`.

Paste the error and the function and I'll find it.

You

Ive been working on code to help me keep track of the budget and when it goes over based on the other buttons and code we have already done, but my original page isn't updating with the "money spent" and "money left". These are the codes Ive added:

```
def total_planned(people: list[Person]) -> float:
    total = 0.0
    for person in people:
        for gift in person.gifts:
            total = total + gift.price
    return total

@route
def save_gift(state: State, name: str, price: float, person_name: str) -> Page:
    if total_planned(state.recipients) + price > state.budget:
        return Page(state, [
            Header("Over Budget Warning"),
            "Adding " + name + " ($" + str(price) + ") will put you over your budget of $" + str(state.budget) + ".",
            Button("Add Anyway", confirm_gift, [Argument("name", name),
                                                Argument("price", price),
                                                Argument("person_name", person_name)]),
            Button("Cancel", view_person, [Argument("person_name", person_name)])
        ])
    return confirm_gift(state, name, price, person_name)

@route
def confirm_gift(state: State, name: str, price: float, person_name: str) -> Page:
    for person in state.recipients:
        if person.name == person_name:
            person.gifts.append(Gift(name, price, False))
            state.total_spent = total_all(state.recipients)
    return view_person(state, person_name)
```

Claude

The likely culprit is that `confirm_gift` calls `total_all`, but the helper you added is named `total_planned`. If `total_all` doesn't exist in your file anymore, that line errors and `total_spent` never updates. Use the function you actually have:

```
state.total_spent = total_planned(state.recipients)
```

Key points: - `index` shows `state.total_spent`, so that value has to be updated every time a gift is added. `confirm_gift` is the right place for it. - Since `total_planned` already adds up every gift, you could skip storing `total_spent` and compute it in `index` instead, so it can't go stale:

```
spent = total_planned(state.recipients)
"Spent so far: $" + str(spent),
"Money left: $" + str(state.budget - spent),
```

- Your over-budget check in `save_gift` looks correct.
- If the numbers still don't change, restart the app and add a gift again. Old saved state can hide the fix.

If you get an error message, paste it and I'll take a look.

You

well should I changed the index route or just add that code somewhere

Claude

Change the `index` route. You're replacing the old lines, not adding new ones.

```
@route
def index(state: State) -> Page:
    spent = total_planned(state.recipients)
    content = [
        Header("Gift Tracker"),
        "Total budget: $" + str(state.budget),
        "Spent so far: $" + str(spent),
        "Money left: $" + str(state.budget - spent),
        "Set your overall budget:",
        TextBox("new_budget", str(state.budget)),
        Button("Save Budget", save_budget),
        Button("View People!", view_people)
    ]
    if spent > state.budget:
        content.append("Warning: over budget by $" + str(spent - state.budget))
    return Page(state, content)
```

Key points: - `spent` is recalculated every time the page loads, so it's always current. - With this, `state.total_spent` is no longer needed on this page. You can leave the line in `confirm_gift` or delete it, and it won't break anything either way. - Make sure `total_planned` is defined above (or anywhere in the file) so `index` can find it.

You

okay thanks that actually worked

Claude

Glad it worked! If you want a next step, you could add a per-person total on each person's page using `total_gifts`, or sort the gifts by price. Just let me know what you want to tackle.

You

I just realized that I only have 3 state fields but I need 4. What is something I can add to my state field that will add value to my project?

Claude

A few ideas that would add real value:

- `message: str`: shows feedback like "Please enter a valid price" or "Gift added!" on your pages. Useful for error handling.
- `show_unbought_only: bool`: lets a button filter gift lists to only what you still need to buy.
- `per_person_limit: float`: a spending limit for each person, which fits Secret Santas, and you could warn when a person goes over it.

- `deadline: str`: the date you need everything bought by, shown on the home page.

My pick is `per_person_limit`. It builds on code you already have (the sum pattern and the over-budget warning) and makes the project more useful.

Since you now calculate spending with `total_planned`, `total_spent` is redundant, so you could swap it out for the new field instead of keeping both.

If you add a field, remember to update `start_server(State(...))` and every place you create a `State`.

You

I think I like the per_person_limit because I do have a lot of secret santas this year. How could we do that?

Claude

Here's how to add it in five steps.

1. Add the field to `State` and `start_server`:

```
@dataclass
class State:
    budget: float
    total_spent: float
    recipients: list[Person]
    per_person_limit: float

start_server(State(0.0, 0.0, [], 0.0))
```

2. Add a limit box to `index`. Put these in your `content` list:

```
"Limit per person: $" + str(state.per_person_limit),
TextBox("new_limit", str(state.per_person_limit)),
Button("Save Limit", save_limit),

@route
def save_limit(state: State, new_limit: str) -> Page:
    state.per_person_limit = float(new_limit)
    return index(state)
```

3. Add a helper that sums one person's gifts:

```
def total_gifts(gifts: list[Gift]) -> float:
    total = 0.0
    for gift in gifts:
        total = total + gift.price
    return total
```

4. Check the limit in `save_gift`. Add this at the top, before your existing budget check:

```
for person in state.recipients:
    if person.name == person_name:
        if state.per_person_limit > 0 and total_gifts(person.gifts) + price > state.per_person_limit:
            return Page(state, [
                Header("Over Limit Warning"),
                person_name + "'s gifts would go over the $" + str(state.per_person_limit) + " limit.",
                Button("Add Anyway", confirm_gift, [Argument("name", name),
                                                    Argument("price", price),
                                                    Argument("person_name", person_name)]),
                Button("Cancel", view_person, [Argument("person_name", person_name)])
            ])
```

5. Show each person's total in `view_person`. Add this before the buttons:

```
content.append("Total for " + person.name + ": $" + str(total_gifts(person.gifts)))
```

Key points: - A limit of `0.0` means "no limit," so nothing breaks before you set one. - If you also want the Secret Santa limit visible as you go, you could add `"Limit: $" + str(state.per_person_limit)` to `view_person` as well.

You

I added these things and my code does work properly, but I do want to make ti that each person can have a set limit, not all the same

Claude

Use the `total` field in `Person` that you aren't using yet. Rename it to `limit` and store each person's limit there.

1. Update the dataclass:

```
@dataclass
class Person:
    name: str
    gifts: list[Gift]
    limit: float
```

2. Ask for the limit when adding a person. Your `per_person_limit` in `State` becomes the default value in the box:

```
@route
def add_reciprient(state: State) -> Page:
    return Page(state, [
        Header("Add Recipient"),
        "Name:",
        TextBox("name"),
        "Limit:",
        TextBox("limit", str(state.per_person_limit)),
        Button("Save", save_reciprient),
        Button("Cancel", view_people)
    ])

@route
def save_reciprient(state: State, name: str, limit: float) -> Page:
    state.recipients.append(Person(name, [], limit))
    return view_people(state)
```

3. Use each person's limit in `save_gift`. Change the check to:

```
if person.limit > 0 and total_gifts(person.gifts) + price > person.limit:
```

and in the warning text, use `str(person.limit)` instead of `str(state.per_person_limit)`.

4. Show it in `view_person`:

```
content.append("Limit for " + person.name + ": $" + str(person.limit))
```

Key points: - `State` keeps all 4 fields. `per_person_limit` now means "the default limit for new people," so you could label it that way on `index`. - Anywhere else you create a `Person(...)`, make sure the third value is the limit. - Restart the app after changing the dataclass so old saved people don't cause errors.

Want a button on each person's page to change their limit later?

You

Your code used the name 'person', but that name has not been created yet (or was misspelled).

request.route_execution_failed: Error while processing request for URL 'add_reciprient': name 'person' is not defined it

Claude

The error says `add_reciprient`

You

it gave me this error message

Claude

That error means `add_reciprient` uses the name `person`, but `person` only exists inside loops in other routes (like `view_person`). It looks like a line meant for another route ended up in this one.

Look inside `add_reciprient` for any line mentioning `person`. The most likely culprit is this one, which belongs in `view_person`:

```
content.append("Limit for " + person.name + ": $" + str(person.limit))
```

`add_reciprient` shouldn't use `person` at all. It should look exactly like this:

```
@route
def add_reciprient(state: State) -> Page:
    return Page(state, [
        Header("Add Recipient"),
        "Name:",
        TextBox("name"),
        "Limit:",
        TextBox("limit", str(state.per_person_limit)),
        Button("Save", save_reciprient),
        Button("Cancel", view_people)
    ])
```

Move the "Limit for..." line into `view_person`, inside the `if person.name == person_name:` block.

If it still fails, paste your `add_reciprient` and I'll spot it.

You

[You attached your full project file.]

This is what I have so far.

You

Can you please help me document the save_gift function and the view_person function since I do understand what they do, but theyre getting very long

Claude

Here are both functions with docstrings and a few comments in the design recipe style. The logic is unchanged.

[Abridged: the full docstrings for view_person and save_gift appear in the final code listing at the end of this document. In this reply I abbreviated the two warning pages in save_gift with placeholder "[...]" lines.]

Replace each `[...]` with your existing warning page code. I shortened them here only to save space.

Since these functions are getting long, you could move each warning page into its own small route (like `over_limit_page` and `over_budget_page`). That would make `save_gift` much shorter and easier to read. I can show you how if you want.

You

Im getting an error

Status error, Severity error, Category request, Recoverable yes, action link, url save_gift, name Electronic, price 10.0, person_name Colin

request.route_execution_failed: Error while processing request for URL 'save_gift': The content of a Page must be a list of strings, numbers, booleans, or components. Found ellipsis at index 0 instead.

Claude

The `[...]` in my example was a placeholder, and it ended up in your code as a literal ellipsis. Drafter reads `...` as content, which isn't allowed. Here is the full `save_gift` with your real warning pages filled in:

[The corrected, full save_gift function with its docstring is included in the final code listing at the end of this document.]

Search your file for `...` to make sure none are left in `view_person` or anywhere else.

You

can you write me some thorough unit tests for each route so I can verify that they all work

Claude

[Abridged: Claude wrote bakery assert_equal unit tests for the helper functions and for every route (index, save_budget, save_limit, add_reciprient, save_reciprient, view_people, view_person, toggle_bought, add_gift, confirm_gift, save_gift). The complete tests are in the final code listing at the end of this document.]

Key points: - Run the file and only failures print, so no output means everything passed. Tests need to be above `start_server`. - Because routes mutate the `State` they receive, each test passes its own freshly built state, and the expected `Page` uses a separate one. - Note that `index` shows "Planned" for all gifts and "Spent so far" for bought gifts only. The tests reflect your current code. - If a Page comparison fails with a confusing difference, check the exact strings (spacing, `$`, `.0`) against your route, since those are the usual cause. - I haven't run these against your Drafter version. If one fails, paste the failure and we can tell whether it's the test or the route.

You

Two more of my projects requirements are :

Claude

Your message cut off after "requirements are:", so the list didn't come through. Could you paste the two requirements again?

You

* Included types in your function headers * Included good docstrings as the first line of your function's body

Claude

Your headers already have types on every parameter and return value, so that requirement is covered. What's missing is docstrings. Only `view_person` and `save_gift` have them so far.

[Abridged: Claude gave a docstring for every function (index, save_budget, save_limit, add_reciprient, save_reciprient, view_people, toggle_bought, add_gift, confirm_gift, person_planned, total_planned, total_gifts, total_all) in Args/Returns style. They appear in the final code listing at the end of this document.]

Your `view_person` and `save_gift` docstrings from earlier already match this style.

Make sure every docstring is the very first thing under its `def` line, before any other code. That's what "first line of the function's body" means in the requirement.

You

can you return the whole code with those doc strings included

Claude

[Abridged: Claude returned the whole project file with every docstring included. This is identical to the final code listing at the end of this document, minus the unit tests section.]

If you added the unit tests, paste them back in just above `start_server(...)`.

You

return the code with the docstrings and the unit tests

Claude

I moved `from bakery import assert_equal` up into the imports. I haven't run the tests, so if one fails, paste the failure and we can tell whether it's the test or the route.

Complete code (docstrings and unit tests):

```
from dataclasses import dataclass
from drafter import *
from bakery import assert_equal

@dataclass
class Gift:
    name: str
    price: float
    is_bought: bool

@dataclass
```

```
class Person:
    name: str
    gifts: list[Gift]
    limit: float

@dataclass
class State:
    budget: float
    total_spent: float
    recipients: list[Person]
    per_person_limit: float

@route
def index(state: State) -> Page:
    """
    Show the home page with the budget, planned and spent totals,
    money left, and a warning if the plan is over budget.

    Args:
        state (State): the current app state

    Returns:
        Page: the Gift Tracker home page
    """
    planned = total_planned(state.recipients)
    content = [
        Header("Gift Tracker"),
        "Total budget: $" + str(state.budget),
        "Planned (all gifts): $" + str(planned),
        "Spent so far (bought): $" + str(state.total_spent),
        "Money left: $" + str(state.budget - planned),
        "Set your overall budget:",
        TextBox("new_budget", str(state.budget)),
        Button("Save Budget", save_budget),
        "Default limit per person: $" + str(state.per_person_limit),
        "Set default limit per person:",
        TextBox("new_limit", str(state.per_person_limit)),
        Button("Save Limit", save_limit),
        Button("View People!", view_people)
    ]
    if planned > state.budget:
        content.append("Warning: over budget by $" + str(planned - state.budget))
    return Page(state, content)

@route
def save_budget(state: State, new_budget: str) -> Page:
    """
    Save the overall budget typed by the user.

    Args:
        state (State): the current app state
        new_budget (str): the budget the user typed in

    Returns:
        Page: the home page with the updated budget
    """
    state.budget = float(new_budget)
    return index(state)

@route
def save_limit(state: State, new_limit: str) -> Page:
    """
    Save the default per-person limit typed by the user.

    Args:
        state (State): the current app state
        new_limit (str): the limit the user typed in

    Returns:
        Page: the home page with the updated default limit
    """
    state.per_person_limit = float(new_limit)
    return index(state)

@route
def add_reciprient(state: State) -> Page:
    """
    Show the form for adding a new recipient and their limit.
```

```
    Args:
        state (State): the current app state

    Returns:
        Page: the Add Recipient form
    """
    return Page(state, [
        Header("Add Recipient"),
        "Name:",
        TextBox("name"),
        "Limit (0 means no limit):",
        TextBox("limit", str(state.per_person_limit)),
        Button("Save", save_reciprient),
        Button("Cancel", view_people)
    ])

@route
def save_reciprient(state: State, name: str, limit: float) -> Page:
    """
    Add a new person with no gifts to the list of recipients.

    Args:
        state (State): the current app state
        name (str): the new person's name
        limit (float): the new person's spending limit (0 means none)

    Returns:
        Page: the people list including the new person
    """
    state.recipients.append(Person(name, [], limit))
    return view_people(state)

@route
def view_people(state: State) -> Page:
    """
    Show a button for every recipient, plus buttons to add a
    recipient or go back to the home page.

    Args:
        state (State): the current app state

    Returns:
        Page: the people list page
    """
    content = [Header("People")]
    for person in state.recipients:
        content.append(Button(person.name, view_person, [Argument("person_name", person.name)]))
    content.append(Button("Add Recipient", add_reciprient))
    content.append(Button("Back", index))
    return Page(state, content)

@route
def view_person(state: State, person_name: str) -> Page:
    """
    Show one person's page: their limit, their gifts with a bought
    checkbox mark, and buttons to toggle gifts, add a gift, or go back.

    Args:
        state (State): the current app state
        person_name (str): the name of the person to display

    Returns:
        Page: the person's gift page, or the people list if no one
        matches person_name
    """
    # Find the person whose name matches the button that was clicked
    for person in state.recipients:
        if person.name == person_name:
            content = [Header(person.name + "'s Gifts")]
            content.append("Limit for " + person.name + ": $" + str(person.limit))
            # One line and one toggle button per gift
            for gift in person.gifts:
                mark = "[x] " if gift.is_bought else "[ ] "
                content.append(mark + gift.name + ": $" + str(gift.price))
                content.append(Button("Toggle bought", toggle_bought,
                                      [Argument("person_name", person.name),
                                       Argument("gift_name", gift.name)]))
            content.append(Button("Add Gift", add_gift, [Argument("person_name", person.name)]))
```

```
            content.append(Button("Back", view_people))
            return Page(state, content)
    # Fallback: no matching person was found
    return view_people(state)

@route
def toggle_bought(state: State, person_name: str, gift_name: str) -> Page:
    """
    Flip a gift between bought and not bought, then update the
    total spent.

    Args:
        state (State): the current app state
        person_name (str): the person the gift belongs to
        gift_name (str): the gift to toggle

    Returns:
        Page: the person's gift page with the updated checklist
    """
    for person in state.recipients:
        if person.name == person_name:
            for gift in person.gifts:
                if gift.name == gift_name:
                    gift.is_bought = not gift.is_bought
    state.total_spent = total_all(state.recipients)
    return view_person(state, person_name)

@route
def add_gift(state: State, person_name: str) -> Page:
    """
    Show the form for adding a gift for one person.

    Args:
        state (State): the current app state
        person_name (str): the person receiving the gift

    Returns:
        Page: the Add Gift form
    """
    return Page(state, [
        Header("Add Gift for " + person_name),
        "Gift Name:",
        TextBox("name"),
        "Price:",
        TextBox("price"),
        Button("Save", save_gift, [Argument("person_name", person_name)]),
        Button("Cancel", view_person, [Argument("person_name", person_name)])
    ])

@route
def save_gift(state: State, name: str, price: float, person_name: str) -> Page:
    """
    Check a new gift against the person's limit and the overall budget
    before adding it. If either would be exceeded, show a warning page
    that lets the user add the gift anyway or cancel.

    Args:
        state (State): the current app state
        name (str): the name of the new gift
        price (float): the price of the new gift
        person_name (str): the name of the person receiving the gift

    Returns:
        Page: a warning page if a limit would be passed, otherwise the
        result of confirm_gift (the person's updated gift page)
    """
    # Check 1: the person's own limit (0 means no limit)
    for person in state.recipients:
        if person.name == person_name:
            if person.limit > 0 and person_planned(person) + price > person.limit:
                return Page(state, [
                    Header("Over Person Limit"),
                    "Adding " + name + " ($" + str(price) + ") puts " + person_name +
                    " over their $" + str(person.limit) + " limit.",
                    Button("Add Anyway", confirm_gift, [Argument("name", name),
                                                        Argument("price", price),
                                                        Argument("person_name", person_name)]),
                    Button("Cancel", view_person, [Argument("person_name", person_name)])
```

```
                ])
    # Check 2: the overall budget
    if total_planned(state.recipients) + price > state.budget:
        return Page(state, [
            Header("Over Budget Warning"),
            "Adding " + name + " ($" + str(price) + ") will put you over your budget of $" + str(state.budget) + ".",
            Button("Add Anyway", confirm_gift, [Argument("name", name),
                                                Argument("price", price),
                                                Argument("person_name", person_name)]),
            Button("Cancel", view_person, [Argument("person_name", person_name)])
        ])
    # No limits passed, so add the gift
    return confirm_gift(state, name, price, person_name)

@route
def confirm_gift(state: State, name: str, price: float, person_name: str) -> Page:
    """
    Add a gift to a person's list without any limit checks, then
    update the total spent.

    Args:
        state (State): the current app state
        name (str): the gift's name
        price (float): the gift's price
        person_name (str): the person receiving the gift

    Returns:
        Page: the person's gift page including the new gift
    """
    for person in state.recipients:
        if person.name == person_name:
            person.gifts.append(Gift(name, price, False))
    state.total_spent = total_all(state.recipients)
    return view_person(state, person_name)

def person_planned(person: Person) -> float:
    """
    Add up the prices of all of one person's gifts, bought or not.

    Args:
        person (Person): the person whose gifts to total

    Returns:
        float: the total price of the person's gifts
    """
    total = 0.0
    for gift in person.gifts:
        total = total + gift.price
    return total

def total_planned(people: list[Person]) -> float:
    """
    Add up the prices of every gift for every person, bought or not.

    Args:
        people (list[Person]): all of the recipients

    Returns:
        float: the total planned cost
    """
    total = 0.0
    for person in people:
        total = total + person_planned(person)
    return total

def total_gifts(gifts: list[Gift]) -> float:
    """
    Add up the prices of only the gifts marked as bought.

    Args:
        gifts (list[Gift]): the gifts to total

    Returns:
        float: the total cost of the bought gifts
    """
    total = 0.0
    for gift in gifts:
        if gift.is_bought:
```

```
            total = total + gift.price
    return total

def total_all(people: list[Person]) -> float:
    """
    Add up the prices of all bought gifts across every person.

    Args:
        people (list[Person]): all of the recipients

    Returns:
        float: the total amount spent so far
    """
    total = 0.0
    for person in people:
        total = total + total_gifts(person.gifts)
    return total


# ====================== UNIT TESTS ======================

# ---------- helper functions ----------
assert_equal(person_planned(Person("Ann", [], 0.0)), 0.0)
assert_equal(person_planned(Person("Ann", [Gift("Book", 10.0, False), Gift("Pen", 5.0, True)], 0.0)), 15.0)

assert_equal(total_planned([]), 0.0)
assert_equal(total_planned([Person("Ann", [Gift("Book", 10.0, False)], 0.0),
                            Person("Bo", [Gift("Pen", 5.0, True)], 0.0)]), 15.0)

# total_gifts / total_all only count BOUGHT gifts
assert_equal(total_gifts([Gift("Book", 10.0, False), Gift("Pen", 5.0, True)]), 5.0)
assert_equal(total_gifts([]), 0.0)
assert_equal(total_all([Person("Ann", [Gift("Book", 10.0, True)], 0.0),
                        Person("Bo", [Gift("Pen", 5.0, False)], 0.0)]), 10.0)

# ---------- index ----------
assert_equal(
    index(State(100.0, 0.0, [], 0.0)),
    Page(State(100.0, 0.0, [], 0.0), [
        Header("Gift Tracker"),
        "Total budget: $100.0",
        "Planned (all gifts): $0.0",
        "Spent so far (bought): $0.0",
        "Money left: $100.0",
        "Set your overall budget:",
        TextBox("new_budget", "100.0"),
        Button("Save Budget", save_budget),
        "Default limit per person: $0.0",
        "Set default limit per person:",
        TextBox("new_limit", "0.0"),
        Button("Save Limit", save_limit),
        Button("View People!", view_people)
    ]))

# over budget: warning line is added
over = State(50.0, 0.0, [Person("Ann", [Gift("Book", 60.0, False)], 0.0)], 0.0)
assert_equal(
    index(over),
    Page(over, [
        Header("Gift Tracker"),
        "Total budget: $50.0",
        "Planned (all gifts): $60.0",
        "Spent so far (bought): $0.0",
        "Money left: $-10.0",
        "Set your overall budget:",
        TextBox("new_budget", "50.0"),
        Button("Save Budget", save_budget),
        "Default limit per person: $0.0",
        "Set default limit per person:",
        TextBox("new_limit", "0.0"),
        Button("Save Limit", save_limit),
        Button("View People!", view_people),
        "Warning: over budget by $10.0"
    ]))

# ---------- save_budget / save_limit ----------
assert_equal(save_budget(State(0.0, 0.0, [], 0.0), "75.5"),
             index(State(75.5, 0.0, [], 0.0)))
```

```
assert_equal(save_limit(State(0.0, 0.0, [], 0.0), "20.0"),
             index(State(0.0, 0.0, [], 20.0)))

# ---------- add_reciprient ----------
assert_equal(
    add_reciprient(State(0.0, 0.0, [], 25.0)),
    Page(State(0.0, 0.0, [], 25.0), [
        Header("Add Recipient"),
        "Name:",
        TextBox("name"),
        "Limit (0 means no limit):",
        TextBox("limit", "25.0"),
        Button("Save", save_reciprient),
        Button("Cancel", view_people)
    ]))

# ---------- save_reciprient ----------
assert_equal(save_reciprient(State(0.0, 0.0, [], 0.0), "Colin", 20.0),
             view_people(State(0.0, 0.0, [Person("Colin", [], 20.0)], 0.0)))
# adds to the end of an existing list
assert_equal(save_reciprient(State(0.0, 0.0, [Person("Ann", [], 0.0)], 0.0), "Bo", 5.0),
             view_people(State(0.0, 0.0, [Person("Ann", [], 0.0), Person("Bo", [], 5.0)], 0.0)))

# ---------- view_people ----------
assert_equal(
    view_people(State(0.0, 0.0, [], 0.0)),
    Page(State(0.0, 0.0, [], 0.0), [
        Header("People"),
        Button("Add Recipient", add_reciprient),
        Button("Back", index)
    ]))
two = State(0.0, 0.0, [Person("Ann", [], 0.0), Person("Bo", [], 0.0)], 0.0)
assert_equal(
    view_people(two),
    Page(two, [
        Header("People"),
        Button("Ann", view_person, [Argument("person_name", "Ann")]),
        Button("Bo", view_person, [Argument("person_name", "Bo")]),
        Button("Add Recipient", add_reciprient),
        Button("Back", index)
    ]))

# ---------- view_person ----------
s = State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, True), Gift("Pen", 5.0, False)], 20.0)], 0.0)
assert_equal(
    view_person(s, "Colin"),
    Page(s, [
        Header("Colin's Gifts"),
        "Limit for Colin: $20.0",
        "[x] Book: $10.0",
        Button("Toggle bought", toggle_bought, [Argument("person_name", "Colin"),
                                                Argument("gift_name", "Book")]),
        "[ ] Pen: $5.0",
        Button("Toggle bought", toggle_bought, [Argument("person_name", "Colin"),
                                                Argument("gift_name", "Pen")]),
        Button("Add Gift", add_gift, [Argument("person_name", "Colin")]),
        Button("Back", view_people)
    ]))
# person with no gifts
s = State(0.0, 0.0, [Person("Ann", [], 0.0)], 0.0)
assert_equal(
    view_person(s, "Ann"),
    Page(s, [
        Header("Ann's Gifts"),
        "Limit for Ann: $0.0",
        Button("Add Gift", add_gift, [Argument("person_name", "Ann")]),
        Button("Back", view_people)
    ]))
# name not found falls back to the people list
assert_equal(view_person(State(0.0, 0.0, [], 0.0), "Nobody"),
             view_people(State(0.0, 0.0, [], 0.0)))

# ---------- toggle_bought ----------
# unbought -> bought (total_spent updates)
assert_equal(
    toggle_bought(State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 0.0)], 0.0), "Colin", "Book"),
    view_person(State(0.0, 10.0, [Person("Colin", [Gift("Book", 10.0, True)], 0.0)], 0.0), "Colin"))
# bought -> unbought (total_spent goes back down)
```

```
assert_equal(
    toggle_bought(State(0.0, 10.0, [Person("Colin", [Gift("Book", 10.0, True)], 0.0)], 0.0), "Colin", "Book"),
    view_person(State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 0.0)], 0.0), "Colin"))
# only the matching gift changes
assert_equal(
    toggle_bought(State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False), Gift("Pen", 5.0, False)], 0.0)], 0.0), "Colin", "Pen"
    view_person(State(0.0, 5.0, [Person("Colin", [Gift("Book", 10.0, False), Gift("Pen", 5.0, True)], 0.0)], 0.0), "Colin"))
# gift name not found: nothing changes
assert_equal(
    toggle_bought(State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 0.0)], 0.0), "Colin", "Nope"),
    view_person(State(0.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 0.0)], 0.0), "Colin"))

# ---------- add_gift ----------
assert_equal(
    add_gift(State(0.0, 0.0, [], 0.0), "Colin"),
    Page(State(0.0, 0.0, [], 0.0), [
        Header("Add Gift for Colin"),
        "Gift Name:",
        TextBox("name"),
        "Price:",
        TextBox("price"),
        Button("Save", save_gift, [Argument("person_name", "Colin")]),
        Button("Cancel", view_person, [Argument("person_name", "Colin")])
    ]))

# ---------- confirm_gift ----------
# adds the gift; total_spent only counts bought gifts, so it stays 0.0
assert_equal(
    confirm_gift(State(100.0, 0.0, [Person("Colin", [], 0.0)], 0.0), "Pen", 5.0, "Colin"),
    view_person(State(100.0, 0.0, [Person("Colin", [Gift("Pen", 5.0, False)], 0.0)], 0.0), "Colin"))

# ---------- save_gift ----------
# under the limit and budget: gift is added
assert_equal(
    save_gift(State(100.0, 0.0, [Person("Colin", [], 20.0)], 0.0), "Pen", 5.0, "Colin"),
    view_person(State(100.0, 0.0, [Person("Colin", [Gift("Pen", 5.0, False)], 20.0)], 0.0), "Colin"))

# exactly at the person limit is allowed (check uses >)
assert_equal(
    save_gift(State(100.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 20.0)], 0.0), "Pen", 10.0, "Colin"),
    view_person(State(100.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False), Gift("Pen", 10.0, False)], 20.0)], 0.0), "Colin"))

# limit of 0 means no limit
assert_equal(
    save_gift(State(1000.0, 0.0, [Person("Colin", [], 0.0)], 0.0), "TV", 500.0, "Colin"),
    view_person(State(1000.0, 0.0, [Person("Colin", [Gift("TV", 500.0, False)], 0.0)], 0.0), "Colin"))

# over the person's limit: warning page, state unchanged
s = State(100.0, 0.0, [Person("Colin", [Gift("Book", 15.0, False)], 20.0)], 0.0)
assert_equal(
    save_gift(s, "Pen", 10.0, "Colin"),
    Page(s, [
        Header("Over Person Limit"),
        "Adding Pen ($10.0) puts Colin over their $20.0 limit.",
        Button("Add Anyway", confirm_gift, [Argument("name", "Pen"),
                                            Argument("price", 10.0),
                                            Argument("person_name", "Colin")]),
        Button("Cancel", view_person, [Argument("person_name", "Colin")])
    ]))

# over the overall budget: warning page, state unchanged
s = State(15.0, 0.0, [Person("Colin", [Gift("Book", 10.0, False)], 0.0)], 0.0)
assert_equal(
    save_gift(s, "Pen", 10.0, "Colin"),
    Page(s, [
        Header("Over Budget Warning"),
        "Adding Pen ($10.0) will put you over your budget of $15.0.",
        Button("Add Anyway", confirm_gift, [Argument("name", "Pen"),
                                            Argument("price", 10.0),
                                            Argument("person_name", "Colin")]),
        Button("Cancel", view_person, [Argument("person_name", "Colin")])
    ]))


start_server(State(0.0, 0.0, [], 0.0))
```

You

Another one is that Im using the drafter python library am I doing that? Im not really sure what that is

Claude

Yes, you're using it. Drafter is the Python library that turns your functions into a website, and it's what your whole project is built on.

You can point to these as evidence: - `from drafter import *` imports the library. - `@route` turns a function into a page of your website. - `Page`, `Header`, `Button`, `TextBox`, and `Argument` are Drafter components that build what shows on screen. - `start_server(...)` launches the site with your starting `State`.

If your requirement asks you to show use of Drafter, those are all things you can mention in a comment or in your write-up.

You

how do I add this conversation to the respiratory in GITHUB

Claude

Assuming you mean your GitHub **repository**, there are two steps: save the conversation as a file, then add that file to the repo.

1. Save the conversation - Copy the chat text and paste it into a new file named something like `llm_conversation.md`. - Or, if your assignment accepts a link, use Claude's share option on this chat and paste the link into the file. I can't tell you exactly where that button is, so check the chat's menu.

2. Add it to your repository

On the GitHub website: - Open your repository, click **Add file**, then **Upload files** (or **Create new file** and paste the text). - Click **Commit changes**.

Or from the command line:

```
git add llm_conversation.md
git commit -m "Add LLM conversation"
git push
```

Check your assignment instructions for the exact format they want (full transcript, link, or a short summary of how you used the LLM), since that can vary by instructor.

You

I dont see add file

Claude

The "Add file" button only shows up in certain situations. Check these:

- **You're on the Code tab** of the repository's main page. It appears above the file list, next to the green **Code** button.
- **You're signed in.** If you're logged out, it won't show.
- **You have write access.** If the repo belongs to someone else (a classmate, or your instructor's template), you'll only see it on a copy you own. In that case, make a **fork** or create your own repo.
- **The repo is empty.** A new repo with no files shows links like "creating a new file" or "uploading an existing file" in the middle of the page instead.
- **You're on a phone.** The mobile site hides some buttons. Try "Desktop site" in your browser menu, or use a computer.

If none of that fixes it, tell me whether the repo is yours or from class, and whether you're on a phone or computer, and I can narrow it down.

You

is there a way to save this convo as a pdf

Claude

Yes, the easiest way is your browser's print function:

1. Open this chat on a computer.
2. Press **Ctrl+P** (Windows) or **Cmd+P** (Mac).
3. Change the destination or printer to **Save as PDF**.
4. Click **Save**.

Scroll to the top of the chat first so the whole conversation loads. On a phone, use the browser's share menu and look for a **Print** or **Save as PDF** option.

You can also copy the chat text into a Google Doc or Word file and export that as a PDF.

I can't promise a built-in export button exists, so check the chat's menu to see. If you'd rather, I can make a PDF of a cleaned-up summary of how you used me on this project, which some instructors prefer to a full transcript.

You

I need just this convo as a pdf